UPDATE and ALTER

These two commands are very important, but many beginners confuse them.

One-line Difference

- **UPDATE** → Changes **data inside rows**
- **ALTER** → Changes **table structure/design**

---

1. UPDATE in MySQL

Definition

UPDATE is a **DML (Data Manipulation Language)** command used to modify existing records in a table.

It changes values stored in rows.

Syntax

```
UPDATE table_name
SET column1 = value1,
    column2 = value2
WHERE condition;
```

Example Table: employees

id	name	salary	dept
1	Ravi	30000	IT
2	Neha	25000	HR
3	Arjun	28000	IT

Example 1: Update One Row

```
UPDATE employees
SET salary = 35000
WHERE id = 1;
```

Result:

id	name	salary
1	Ravi	35000

Example 2: Update Multiple Columns

```
UPDATE employees
SET salary = 40000,
    dept = 'Admin'
WHERE id = 2;
```

Example 3: Update Multiple Rows

```
UPDATE employees
SET salary = salary + 5000
WHERE dept = 'IT';
```

IT employees get increment.

Important Warning

```
UPDATE employees
SET salary = 0;
```

Without WHERE, **all rows update**.

Very common mistake.

Internal Working of UPDATE

When MySQL runs:

```
UPDATE employees SET salary=35000 WHERE id=1;
```

Internally:

1. Find matching row using WHERE.
2. Lock row (InnoDB usually row lock).
3. Change old value to new value.
4. Write change in redo/undo logs.
5. Commit transaction.
6. Release lock.

Real Use Cases of UPDATE

- Change employee salary
- Correct phone number
- Mark order as delivered
- Change product price
- Update account status

---

2. ALTER in MySQL

Definition

ALTER is a **DDL (Data Definition Language)** command used to modify table structure.

It changes schema/design, not row data directly.

Syntax

ALTER TABLE table_name

action;

Common ALTER Operations

A. Add New Column

ALTER TABLE employees

ADD email VARCHAR(100);

Now table gets new column.

B. Modify Column Type

ALTER TABLE employees

MODIFY salary DECIMAL(10,2);

Change datatype.

C. Rename Column

ALTER TABLE employees

RENAME COLUMN name TO emp_name;

D. Drop Column

ALTER TABLE employees

DROP COLUMN dept;

Deletes column permanently.

E. Add Primary Key

ALTER TABLE employees

ADD PRIMARY KEY(id);

F. Rename Table

ALTER TABLE employees

RENAME TO staff;

Internal Working of ALTER

Depends on operation.

When you run:

ALTER TABLE employees ADD email VARCHAR(100);

MySQL may:

1. Update metadata.
2. Sometimes rebuild table.
3. Copy data to new structure.
4. Replace old table.
5. Recreate indexes if needed.

Some ALTERs are fast, some expensive.

Why ALTER Can Be Heavy

For large tables (crores of rows):

- may lock table
- may take time

- may use disk space
- may affect production systems

So DBAs plan ALTER carefully.

Real Use Cases of ALTER

- Add new field mobile_no
- Increase name length
- Add index for performance
- Change datatype
- Remove unused columns

---

3. Difference Between UPDATE vs ALTER

Feature	UPDATE	ALTER
Purpose	Change row data	Change table structure
Type	DML	DDL
Affects	Records	Schema
Uses WHERE	Yes	No
Transaction style	Row-level changes	Structural changes
Example	salary = 50000	add column email

Easy Memory Trick

UPDATE = Update Information

Change inside table rows.

ALTER = Alter Shape

Change table design.

Example Together

Suppose employees table:

id | name | salary

Add new column:

ALTER TABLE employees ADD bonus INT;

Now structure changed.

Fill bonus values:

UPDATE employees

SET bonus = 5000

WHERE id = 1;

Now data changed.

Interview-Level Difference

Question:

Can UPDATE add a column?

❌ No

Can ALTER change salary values row by row?

❌ No (not normal purpose)

Performance Difference

UPDATE

If indexed WHERE used:

WHERE id=5

Fast.

Without WHERE:

May scan all rows.

ALTER

Can be costly on large tables because schema change may rebuild table.

Transaction Behavior

UPDATE

Can often rollback if inside transaction.

START TRANSACTION;

UPDATE ...

ROLLBACK;

ALTER

Many ALTER statements auto-commit.

Common Beginner Mistakes

Mistake 1

Using UPDATE without WHERE.

Mistake 2

Using ALTER when wanting to change values.

Mistake 3

Dropping column accidentally.

Practical Example

Need to increase all salaries by 10%

Use:

```
UPDATE employees
SET salary = salary * 1.10;
```

Need to store joining_date column

Use:

```
ALTER TABLE employees
ADD joining_date DATE;
```

Final Summary

UPDATE

Used to modify existing data inside rows.

ALTER

Used to modify table structure/schema.

One-Line Interview Answer

UPDATE changes table data, while ALTER changes table definition.

Practice Questions

1. Add phone column to students table.
2. Change salary of employee id=3.
3. Rename column name to full_name.
4. Increase all IT salaries by 2000.
5. Remove unused column temp_col.

DELETE and TRUNCATE

These commands are commonly confused because all can remove data. But internally and practically they are very different.

Quick One-Line Meaning

- **DELETE** → Remove selected rows (or all rows) from a table
- **TRUNCATE** → Remove **all rows quickly**, reset table to empty
- **DROP** → Remove the entire table itself (structure + data)

Part 1: DELETE in MySQL

Definition

DELETE is a **DML (Data Manipulation Language)** command used to remove rows from a table.

It removes data row-by-row.

Syntax

DELETE FROM table_name

WHERE condition;

Example Table: employees

id	name	dept	salary
1	Ravi	IT	30000
2	Neha	HR	25000
3	Arjun	IT	28000

Example 1: Delete One Row

DELETE FROM employees

WHERE id = 2;

Deletes Neha only.

Example 2: Delete Multiple Rows

DELETE FROM employees

WHERE dept = 'IT';

Deletes Ravi and Arjun.

Example 3: Delete All Rows

DELETE FROM employees;

All rows removed, **table still exists**.

Internal Working of DELETE

When you run:

DELETE FROM employees WHERE id = 2;

MySQL generally:

1. Finds matching row(s)
2. Locks row(s)
3. Marks/removes records
4. Updates indexes
5. Writes undo/redo logs
6. Can rollback if in transaction (InnoDB)

Important Advantage of DELETE

Can use WHERE

This is huge.

You can precisely remove only needed rows.

Real Uses of DELETE

- Remove cancelled orders
- Delete inactive users
- Delete duplicate rows
- Delete test data

Part 2: TRUNCATE in MySQL

Definition

TRUNCATE removes **all rows** from a table quickly.

It keeps table structure but empties table.

Syntax

TRUNCATE TABLE table_name;

Example

TRUNCATE TABLE employees;

Now:

id	name	dept	salary

(no rows)

But table still exists.

Internal Working of TRUNCATE

Unlike DELETE row-by-row, TRUNCATE usually works more like:

1. Drop old data pages
2. Recreate empty table structure
3. Reset metadata

4. Reset AUTO_INCREMENT (usually)

That is why it is much faster.

Why TRUNCATE is Fast

Because it does **not process each row individually**.

For huge tables, this matters a lot.

Real Uses of TRUNCATE

- Clear staging tables in ETL jobs
- Empty temporary tables
- Reset dev/test tables quickly
- Remove all old imported data

Important Limitation of TRUNCATE

No WHERE clause

You cannot do:

TRUNCATE TABLE employees WHERE id=1;

❌ Invalid

TRUNCATE means all rows only.

DELETE vs TRUNCATE Example

Suppose table has 10 crore rows.

DELETE

DELETE FROM logs;

May be slow because row-by-row delete.

TRUNCATE

TRUNCATE TABLE logs;

Much faster.

AUTO_INCREMENT Difference

Suppose IDs were:

1,2,3,4,5

After:

DELETE FROM employees;

Next insert may continue at 6 (depends/settings/history).

After:

TRUNCATE TABLE employees;

Usually next insert starts again at 1.

Transactions / Rollback

DELETE

Often can rollback inside transaction.

START TRANSACTION;

DELETE FROM employees WHERE id=1;

ROLLBACK;

Data returns.

TRUNCATE

Usually acts like DDL / auto-commit behavior.

Rollback support differs by engine/version, but commonly treat as non-row transactional.

Use carefully.

Part 3: DELETE vs TRUNCATE vs DROP

Feature	DELETE	TRUNCATE	DROP
Removes rows	Yes	Yes (all)	Yes
Removes table structure	No	No	Yes
WHERE allowed	Yes	No	No
Deletes selected rows	Yes	No	No
Faster for full clear	Slower	Faster	Fast
Resets AUTO_INCREMENT	Usually No	Usually Yes	N/A
Table remains after command	Yes	Yes	No

Feature	DELETE	TRUNCATE	DROP
Type	DML	DDL-like	DDL

DROP Explained

Definition

Deletes the whole table:

- data gone
- structure gone
- indexes gone
- constraints gone

DROP TABLE employees;

Now table no longer exists.

Easy Memory Trick

DELETE = Remove rows carefully

TRUNCATE = Empty table quickly

DROP = Destroy table completely

Practical Company Examples

DELETE

Remove users inactive for 5 years:

DELETE FROM users

WHERE last_login < '2020-01-01';

TRUNCATE

Nightly clear staging table:

TRUNCATE TABLE stage_sales;

DROP

Remove obsolete temp table:

DROP TABLE old_backup_2022;

Common Beginner Mistakes

Mistake 1

Using DELETE without WHERE unintentionally:

DELETE FROM employees;

Deletes all rows.

Mistake 2

Using TRUNCATE when only few rows should be removed.

Mistake 3

Using DROP when wanting only data removed.

Interview Answer (Strong)

DELETE removes rows and supports WHERE.
TRUNCATE removes all rows faster and resets table state.
DROP removes the entire table including structure.

Internal Perspective

DELETE

Touches rows + indexes + logs row by row.

TRUNCATE

Reinitializes storage structure.

DROP

Removes metadata object itself.

Final Summary

Use:

- **DELETE** → when you need control/filtering
- **TRUNCATE** → when you need empty table fast
- **DROP** → when table no longer needed

Practice Questions

1. Remove only HR employees.
2. Empty logs table quickly.
3. Remove products table permanently.

4. Delete rows where salary < 20000.
5. Reset staging table daily.

Smart Advice

In production, always pause before running:

DELETE ...

TRUNCATE ...

DROP ...

Wrong command can be expensive.